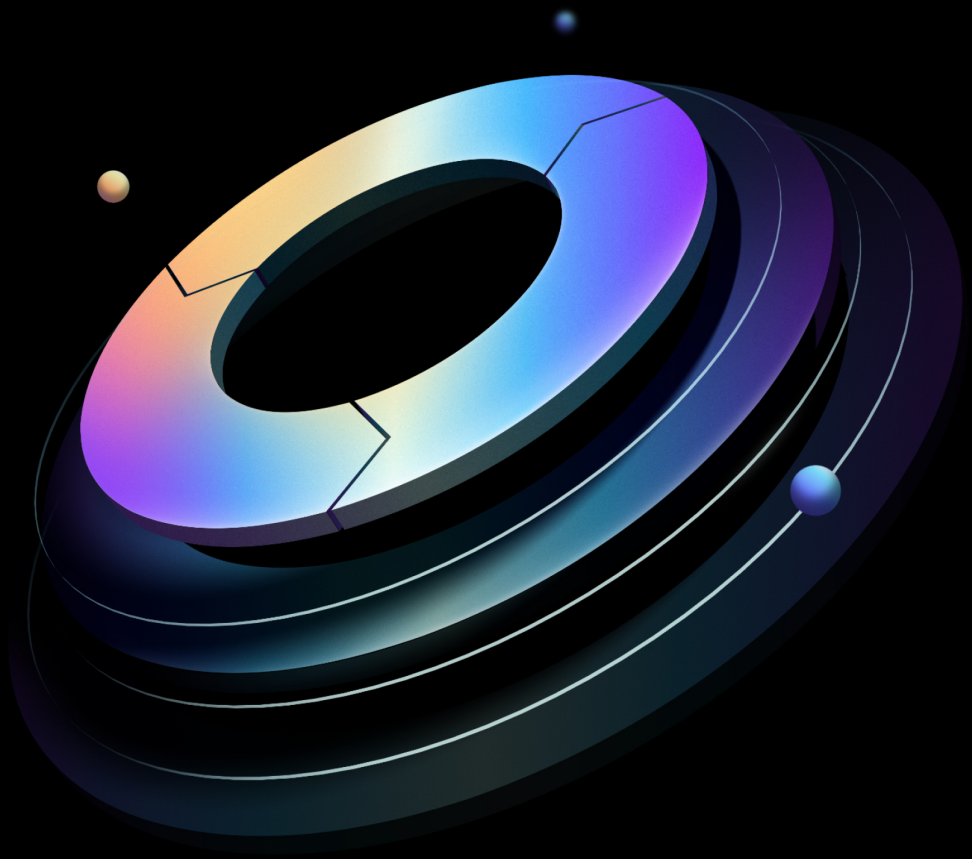
HashiCorp

Nomad Installation Guide

Built from commit 3a6a26c83c

HashiCorp | Validated Designs

09 November 2026



HashiCorp

Contents

1 Introduction **5**
1.1 Why use HashiCorp Validated Designs? . . . 5
1.2 Audience . . . 5
1.3 Supported platforms and versions . . . 6
1.4 Definitions and concepts . . . 6
1.5 Organizational requirements . . . 9

2 Prerequisites and personnel **9**
2.1 Supported platforms and versions . . . 9
2.2 Required teams . . . 10

3 Architecture **10**
3.1 Cluster . . . 10
3.1.1 Clustering . . . 11
3.2 Load balancing . . . 13
3.2.1 TLS . . . 13
3.3 Key management . . . 14
3.4 Backup snapshot storage . . . 14
3.5 Audit . . . 14
3.6 Cluster architecture summary . . . 14

4 Detailed design **16**
4.1 Servers . . . 16
4.2 Networking . . . 17
4.2.1 Availability Zones . . . 18
4.2.2 Network connectivity . . . 19
4.2.3 Traffic Encryption . . . 20
4.2.4 DNS . . . 21
4.2.5 Load balancing . . . 21
4.2.6 TLS passthrough . . . 21
4.2.7 Health check . . . 22
4.3 Software . . . 22
4.4 Nomad License . . . 22
4.5 Configuration . . . 22

4.6 Operating systems . . . 23
4.7 Node Pools . . . 23
4.7.1 Example Use Cases: . . . 24
4.8 Multi-region federation . . . 24
4.8.1 Example Use Cases: . . . 25

5 Cloud deployment (AWS, Azure, GCP virtual machines) 25
5.1 Architectural summary . . . 25
5.1.1 Server naming and IP addressing . . . 25
5.1.2 Certificates . . . 26
5.1.3 Firewall rules . . . 27
5.1.4 Load balancing the API and UI . . . 27
5.1.5 Download and install the Nomad CLI . . . 28
5.1.6 Obtain the Nomad Enterprise license file . . . 28
5.2 Platform-specific guidance . . . 28
5.2.1 Deployment considerations . . . 30
5.3 Deployment process . . . 31
5.3.1 Create and connect to your instances . . . 31
5.3.2 Download and install Nomad . . . 31
5.3.3 Starting Nomad . . . 35
5.3.4 Validate the cluster is bootstrapped and functioning . . . 36
5.3.5 Initialize ACL system . . . 36
5.3.6 Interact with your Nomad cluster . . . 37
5.3.7 Nomad clients . . . 37
5.3.8 Post-installation tasks . . . 40

6 Manual installation of Nomad Enterprise 40
6.1 Preparation . . . 40
6.1.1 Server requirements . . . 40
6.1.2 Client requirements . . . 41
6.1.3 Network requirements . . . 41
6.1.4 Certificates . . . 42
6.1.5 Load balancing the API and UI . . . 43
6.1.6 Auto-join . . . 43
6.1.7 Obtain the Nomad Enterprise license file . . . 44

6.1.8 Download and install the Nomad command-line tool . . . 44
6.2 Deployment process . . . 44
6.2.1 Infrastructure bootstrapping . . . 44
6.2.2 Download and install Nomad . . . 45
6.2.3 Create necessary directories . . . 46
6.2.4 Copy TLS certificates . . . 46
6.2.5 Copy license file . . . 46
6.2.6 Create gossip encryption key . . . 46
6.2.7 Create Nomad configuration file . . . 46
6.2.8 Starting Nomad . . . 48

7 Nomad servers should be run as the nomad user. 48

8 Nomad clients should be run as root 48
8.0.1 Validate the cluster . . . 48
8.0.2 Initialize ACL system . . . 49
8.0.3 Interact with your Nomad cluster . . . 49
8.0.4 Client installation . . . 50
8.1 Post-installation tasks . . . 51

9 Windows clients 52
9.1 Path conventions . . . 52
9.2 Allocation isolation . . . 52
9.3 Docker and Podman orchestration . . . 52
9.3.1 Allowed caps . . . 53
9.4 IPv6 . . . 53
9.5 Logs . . . 53
9.6 Volumes . . . 54
9.7 WSL2 networking . . . 54
9.8 Example Nomad Windows client configuration . . . 55

10 Post-installation configuration 55

11 Next steps 56

12 Changelog 56
12.1 2026-08 . . . 56

12.2 2026-01 56
12.3 2025-10 57
12.4 2025-07 57
12.5 2025-06 57
12.6 2025-05 57
12.7 2024-11 57

13 Credits **57**

1 Introduction

Note

These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

1.2 Audience

This document is intended primarily for practitioners (including platform, networking, identity, and InfoSec teams) who want to deploy Nomad Enterprise clusters on-premises or on cloud infrastructure.

After using the Installation Guide to successfully deploy your Nomad Enterprise infrastructure, Nomad operators should refer to the Administration Guide. Using the Administration Guide, Nomad operators can learn how to integrate application platforms, configure machine and user access, centralize secrets, perform upgrades, and more.

1.3 Supported platforms and versions

As of HVD Document Version 1.0, the Installation Guide supports deploying Nomad Enterprise on the following compute platforms:

- Bare-metal or VM-based servers in a private datacenter
- AWS on EC2 instances

This guide applies to Nomad Enterprise versions `nomad_1.9.3+ent` and later, and can be downloaded from HashiCorp Releases or the HashiCorp Developer page.

While Nomad Enterprise is officially supported for a number of different operating systems and processor architectures, this guide primarily uses `linux/amd64` within its examples, unless otherwise specified.

1.4 Definitions and concepts

This documentation intentionally uses technology agnostic terminology. However, there are some terms which do not translate perfectly between providers. The following are the definitions of terms this document uses:

Term	Definition
Availability zone	An availability zone (or AZ) is a single network failure domain that hosts part or all of a Nomad cluster. Examples of availability zones include:- an individual datacenter- an air-gapped rack in a datacenter- an “Availability Zone” in AWS or Azure- a “Zone” in GCP
Region	A physical location in a distinct geographic area containing one or more datacenters.
Server node	A Nomad server node is a node that runs the Nomad process in `server` mode. Server nodes are responsible for managing the cluster state, scheduling jobs, and handling client requests.

Term	Definition
Leader node	The server node in a Nomad cluster that is responsible for managing the cluster state, scheduling jobs, and handling client requests.
Follower/standby node	Server nodes that forward requests to the leader node. They also service read requests when Redundancy Zones are enabled (see below). Standby nodes vote to elect a new active node if the current one is unavailable after an interval threshold.
Client node	A Nomad client node is a node that runs the Nomad process in `client` mode. Client nodes are responsible for running tasks and reporting their status back to the Nomad servers.
Quorum	Quorum consists of a majority of voting nodes in a Nomad cluster: $(N+1)/2$, where N = the total number of voting nodes. For example, if there are 3 voting nodes, we would need 2 nodes to form a quorum.Quorum ensures the integrity and consistency of the cluster's state. If a quorum of nodes is unavailable for any reason, the cluster becomes unavailable, meaning no client requests can be processed until quorum is once again achieved. *Note:* workloads already scheduled will continue to run.

Term	Definition
Redundancy zone (RZ)	Redundancy zones are a feature of the Autopilot. A redundancy zone consists of multiple nodes in the same "zone" (user-defined), with Autopilot automatically keeping only one as voting, and the others as non-voting, per zone. This provides additional horizontal read scalability as well as increased fault tolerance for the Nomad cluster. If a voting node fails, the non-voting node in the same zone will be automatically promoted to a voter to preserve quorum.
Cluster	Multiple Nomad servers nodes operating together in a high-availability mode. Each node contains an identical set of replicated data storage.
Datacenter	A logical grouping of nodes in a Nomad cluster. This is an entirely user-defined and potentially arbitrary grouping, and doesn't have to reflect a physical datacenter.
Node pools	A collection of nodes within a `datacenter` grouping, which can have custom scheduling parameters and which can be assigned to corresponding Nomad namespaces. Best used for grouping together nodes with special types, such as GPU, ARM-based, high performance, special networking, etc.
Tokens	Tokens are the core method for authentication within Nomad.

1.5 Organizational requirements

Most tasks within this guide will be owned by a single team, who is responsible for deploying shared infrastructure services. This is sometimes referred to as a *platform team*. However, a successful deployment of Nomad Enterprise requires collaboration across various functions within the organization. The platform team may interface with various other teams, such as networking, information security, or identity teams, to perform tasks such as allocating IP space, managing certificates, defining identity roles and permissions, or overseeing DNS hosted zones and records. In cases where the infrastructure resides on a public cloud, these functions might converge under a unified cloud team.

The primary platform team should thoroughly review the Installation Guide in order to identify any teams they may be dependent on or may require approval from for the tasks required as part of the deployment. The platform team should appoint a project lead to oversee the deployment process and ensure effective communication with dependent teams and functions.

2 Prerequisites and personnel

This page summarizes the deployment prerequisites and teams needed to implement the Nomad Enterprise Installation Guide.

2.1 Supported platforms and versions

As of HVD Document Version 1.0, the Installation Guide supports deploying Nomad Enterprise on the following compute platforms:

- Bare-metal or VM-based servers in a private datacenter
- AWS on EC2 instances

This guide applies to Nomad Enterprise versions `nomad_1.9.3+ent` and later, and can be downloaded from HashiCorp Releases or the HashiCorp Developer page.

While Nomad Enterprise is officially supported for a number of different operating systems and processor architectures, this guide primarily uses `linux/amd64` within its examples, unless otherwise specified.

2.2 Required teams

Most tasks within this guide will be owned by a single team, who is responsible for deploying shared infrastructure services. This is sometimes referred to as a *platform team*. However, a successful deployment of Nomad Enterprise requires collaboration across various functions within the organization. The platform team may interface with various other teams, such as networking, information security, or identity teams, to perform tasks such as allocating IP space, managing certificates, defining identity roles and permissions, or overseeing DNS hosted zones and records. In cases where the infrastructure resides on a public cloud, these functions might converge under a unified cloud team.

The primary platform team should thoroughly review the Installation Guide in order to identify any teams they may be dependent on or may require approval from for the tasks required as part of the deployment. The platform team should appoint a project lead to oversee the deployment process and ensure effective communication with dependent teams and functions.

3 Architecture

In this section, we will explore the architecture of the HashiCorp Validated Design for Nomad Enterprise. This architecture is designed to provide a highly-available, scalable, and secure Nomad Enterprise deployment that is suitable for production workloads, and is intended to be used as a starting point for your own Nomad Enterprise deployments. First, we will discuss the components that make up a single Nomad cluster and its related infrastructure. We will then add a secondary Nomad cluster, culminating in a multi-cluster architecture.

The primary components that make up a single Nomad cluster are:

- Cluster of Nomad servers
- Load balancer
- Snapshot Storage (Backups)
- Nomad clients

3.1 Cluster

Nomad uses a server/client architecture. Nomad servers are hosting HTTP(S), Serf and RPC endpoints. Throught these endpoints, operators and Nomad clients can communicate with the Nomad

servers.

The Nomad servers are responsible for scheduling and managing jobs, while the Nomad clients are responsible for executing the tasks that make up those jobs.

3.1.1 Clustering

Nomad multi-node deployment is designed for high-availability. This protects against outages by configuring multiple Nomad servers to operate as a single Nomad cluster. To ensure this high-availability, nodes within a cluster should always be spread across multiple availability zones.

The cluster is bootstrapped during the initialization process. Initial bootstrapping results in a cluster of desired count of nodes. One of the nodes is being elected as the *leader*, while the other nodes accept the role of the *followers*). Once a follower node has successfully joined the cluster, data from the leader will begin to replicate to it.

In a Nomad cluster, only the *leader* performs the scheduling. Follower nodes can perform *reads* against storage and forward *writes* to the leader node.

A Nomad cluster must maintain quorum in order to continue servicing requests. Because of this, nodes should be spread across availability zones to increase fault tolerance.

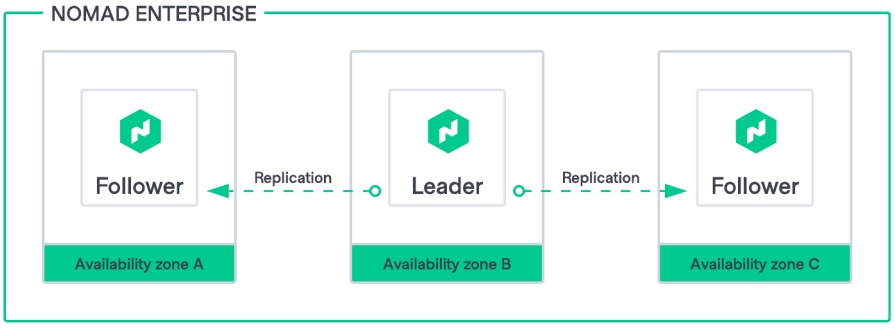


Figure 1: HVD Nomad HA 3 nodes diagram

Voting nodes vs non-voting nodes

By default, standby nodes are also voters. If the active node becomes unavailable, a new active node is chosen by and among the remaining standby nodes. However, standby nodes can also be specified as *non-voters*.

Non-voting nodes also act as data replication targets, but do not contribute to the quorum count.

Autopilot

A non-voting node can be be promoted to a voting node through the use of Nomad Enterprise's Autopilot functionality. In addition to this, Autopilot also enables several automated workflows for managing Nomad clusters, including Server Stabilization, Automated Upgrades, and Redundancy Zones.

For more information, see the HashiCorp Documentation on Nomad Enterprise Autopilot.

Redundancy zones

Using Nomad Enterprise Redundancy Zones, Nomad cluster leverages a pair of nodes in each availability zone, with one node configured as a voter and the other as a non-voter. If one voter becomes unavailable (regardless of whether it is an active node or a standby node), Autopilot will promote its non-voting partner to take its place as a voter.

This architecture gives high availability both within an availability zone as well as across availability zones, ensuring that `(n+1)/2` redundancy is maintained at all times.

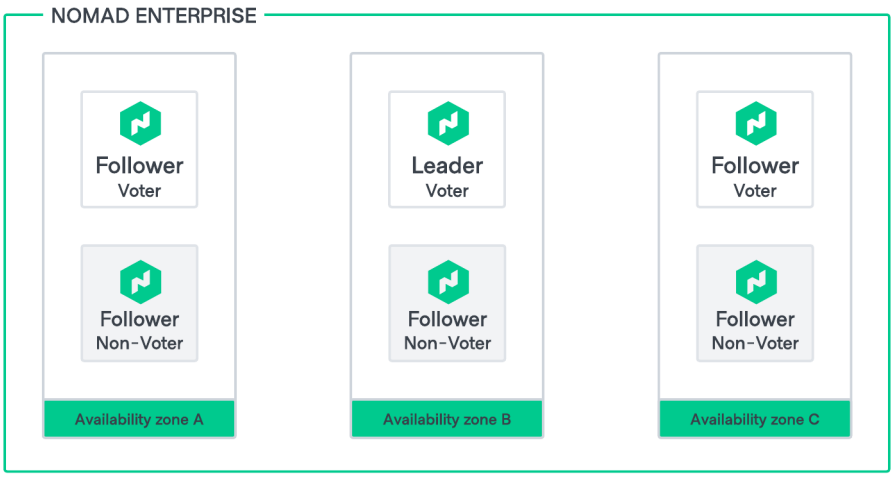


Figure 2: HVD Nomad HA 6 nodes redundancy zones diagram

3.2 Load balancing

Use a layer 4 load balancer capable of handling TCP traffic to direct Nomad operator requests to HTTPS API port of the server nodes within your Nomad cluster. Any request sent to a standby node that requires an active node to process (e.g., job run/dispatch) will be proxied by the standby node to the active node.

Nomad server nodes also perform server-to-server communication to other nodes in the cluster over 4648/`tcp`. This is used for functions critical to the integrity of the cluster itself, such as data replication or active node election. Unlike operator requests, server-to-server traffic should *not* be load-balanced. Instead, each node must be able to communicate directly with all other nodes in the cluster.

> Tip
>
> You should enforce encrypted communication between Nomad servers by enabling gossip encryption. This communication is internal to Nomad and does not require additional certificates or management.

3.2.1 TLS

To ensure the integrity and privacy of transmitted secrets, communications between Nomad operators and Nomad servers should be secured using TLS certificates that you trust.

Do not manage the TLS certificate for the Nomad cluster on the load balancer.

Instead, the certificate chain (i.e., a public certificate, private key, and certificate authority bundle from a trusted certificate authority) should be distributed onto the Nomad servers themselves, and the load balancer should be configured to pass through TLS connections to Nomad nodes directly.

Terminating TLS connections at the Nomad servers offers enhanced security by ensuring that a client request remains encrypted end-to-end, reducing the attack surface for potential eavesdropping or data tampering.

> Warning
>
> Avoid using self-signed certificates for production Nomad workloads.

3.3 Key management

Nomad servers maintain an encryption keyring used to encrypt Variables and sign task workload identities.

You should configure Nomad servers to use an external KMS or Vault transit encryption to wrap the key material using the keyring configuration block.

3.4 Backup snapshot storage

Automated snapshots should be saved to a storage medium, separate from the storage used for the Nomad data directories. Snapshot storage does not require the same high IO-throughput requirements as the rest of Nomad, but it should be durable enough to ensure fast retrieval and restoration of backups when needed. To satisfy this requirement, you can use an object storage system, such as AWS S3, or a network-mounted filesystem, such as NFS.

Configuring the snapshots themselves, their frequency, and other operational considerations can be found in Nomad: Administration Guide.

3.5 Audit

Audit collects and keeps a detailed log of all Nomad requests and corresponding responses. Nomad operators can use these logs to ensure that Nomad is being used in a compliant manner, and to provide an audit trail for forensic analysis in the event of a security incident.

To ensure the availability of audit log data, configure Nomad to use multiple audit destinations. Writing an additional copy of audit log data to a separate device can help prevent data loss in the event one device becomes unavailable, and can also help guard against data tampering.

3.6 Cluster architecture summary

To summarize, the HashiCorp Validated Design for a Nomad Enterprise cluster architecture is made up of the following components:

- 6 Nomad servers, deployed across three availability zones within a geographic region
- Using a redundancy zones topology, each node is configured as a voter or non-voter, with one of each per availability zone

- Locally-attached, high-IO storage for Nomad's Storage
- A layer 4 (TCP) load balancer, with TLS certificates distributed to the Nomad nodes
- An object storage or network file system destination for storing automated snapshots
- Audit logs, configured to write to multiple destinations

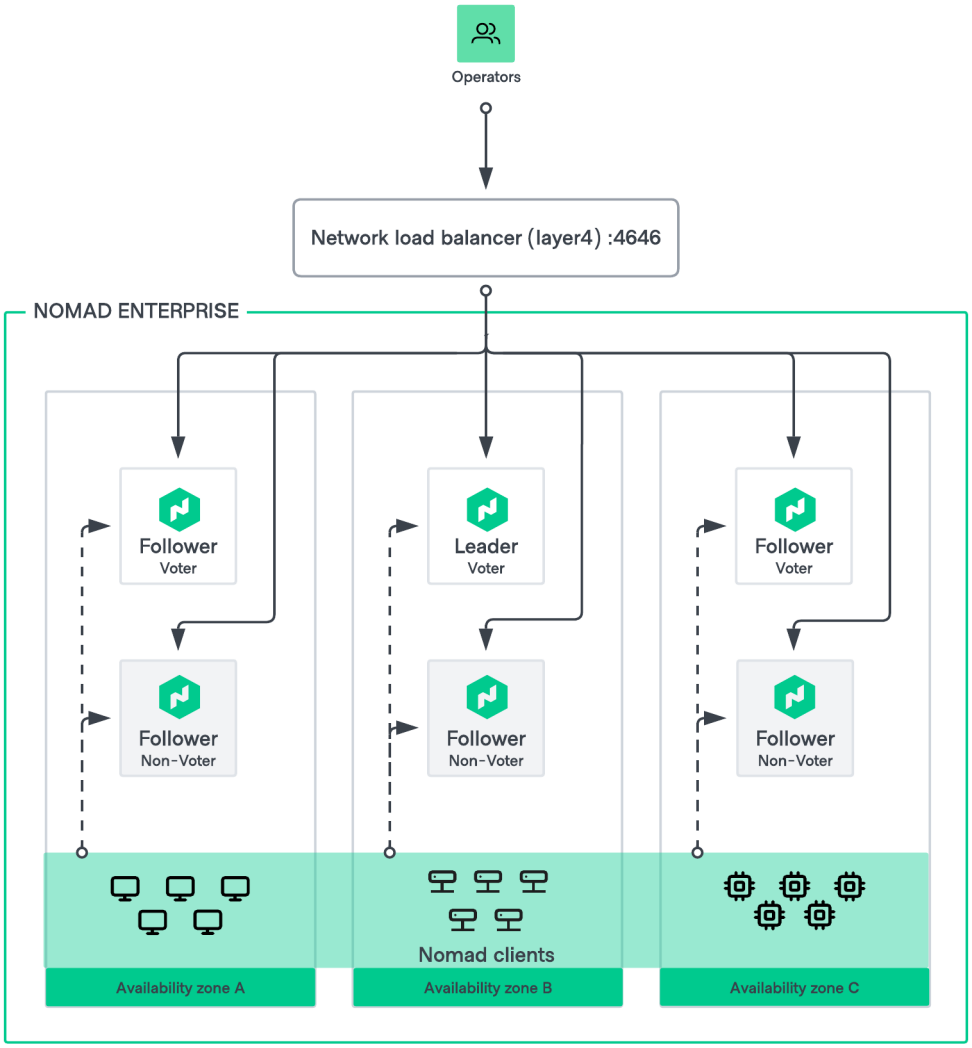


Figure 3: HVD Nomad cluster architecture diagram

4 Detailed design

This section takes the recommended architecture from the above section and provides more detail on each component of the architecture. Carefully review this section to identify all technical and personnel requirements before moving on to implementation.

4.1 Servers

This section contains specific hardware capacity recommendations, network requirements, and additional infrastructure considerations.

Sizing

Every hosting environment is different, and every customer's Nomad usage profile is different. These recommendations should only serve as a starting point, from which each customer's operations staff may observe and adjust to meet the unique needs of each deployment. For more information, refer to the Requirements section of the documentation.

Sizing recommendations have been divided into two common cluster sizes: small, and large. Small clusters would be appropriate for most initial production deployments or for development and testing environments. Large clusters are production environments with a consistently high workload. That might be a large number of deployments, a large number of system or batch jobs, or a combination of the two.

Size	CPU	Memory	Disk Capacity	Disk IO	Disk Throughput
Small	2-4 cores	8-16 GB RAM	100+ GB	3000+ IOPS	75+ MB/s
Large	8-16 cores	32-64 GB RAM	200+ GB	10000+ IOPS	250+ MB/s

For a mapping of these requirements to specific instance types for each cloud provider, refer to the cloud-focused sections below.

Hardware considerations

In general, CPU and storage performance requirements will depend on the customer's exact usage profile (e.g., number of running jobs and deployment rate). Memory requirements depend on the number of running jobs and should be sized according to that.

> **Note**
>
> For predictable performance on cloud providers, avoid "burstable" CPU and storage options, such as AWS `t2` and `t3` instance types. Performance for these types of instances will degrade rapidly under continuous load.

Nomad servers should have a high-performance hard disk subsystem capable of up to 10000+ IOPS and 250+ MB/s disk throughput. Frequently enough Nomad servers need to flush the cluster state data to disk. The use of slower storage systems will negatively impact performance.

Hashicorp strongly recommends configuring Nomad with audit logging enabled, as well as telemetry for diagnostic purposes. The configuration of these components is covered in Nomad Administration Guide. The impact of the additional storage I/O from audit logging will vary depending on your particular pattern of requests.

> **Tip**
>
> Always write audit logs to a separate disk. This approach reduces the risk of performance bottlenecks that can occur when audit logs compete for resources with other system processes.

> **Tip**
>
> Configure multiple audit sinks to enhance reliability and redundancy. If one sink encounters issues, others continue to function, ensuring continuous, uninterrupted logging. This setup is crucial for maintaining robust audit trails and supporting both effective troubleshooting and security monitoring.

4.2 Networking

Nomad servers require interconnection between AZs and load balancers configured with TLS passthrough to connect operators to clusters.

Note

For security reasons, we recommend not to expose Nomad clusters to the public internet.

Servers interconnectivity requires low latency (sub 10ms) to maintain the cluster quorum, as well as to efficiently route operators write requests from standby nodes to the active node. These nodes are also fronted by Layer 4 Load Balancers, which permit end-to-end TLS communication.

Servers Network rules must allow communication the following ports:

- API (4646/tcp) for operators to communicate with the cluster
- RPC (4647/tcp) for clients (compute nodes) to establish a long lasting connection to servers and receive jobs assignments
- Serf (4648/tcp and 4648/udp) between servers only to maintain the cluster quorum

4.2.1 Availability Zones

This Validated Design uses 3 availability zones, with 2 nodes per availability zone.

In order for cluster members to stay properly in sync, network latency between availability zones should be less than ten milliseconds (10 ms).

4.2.2 Network connectivity

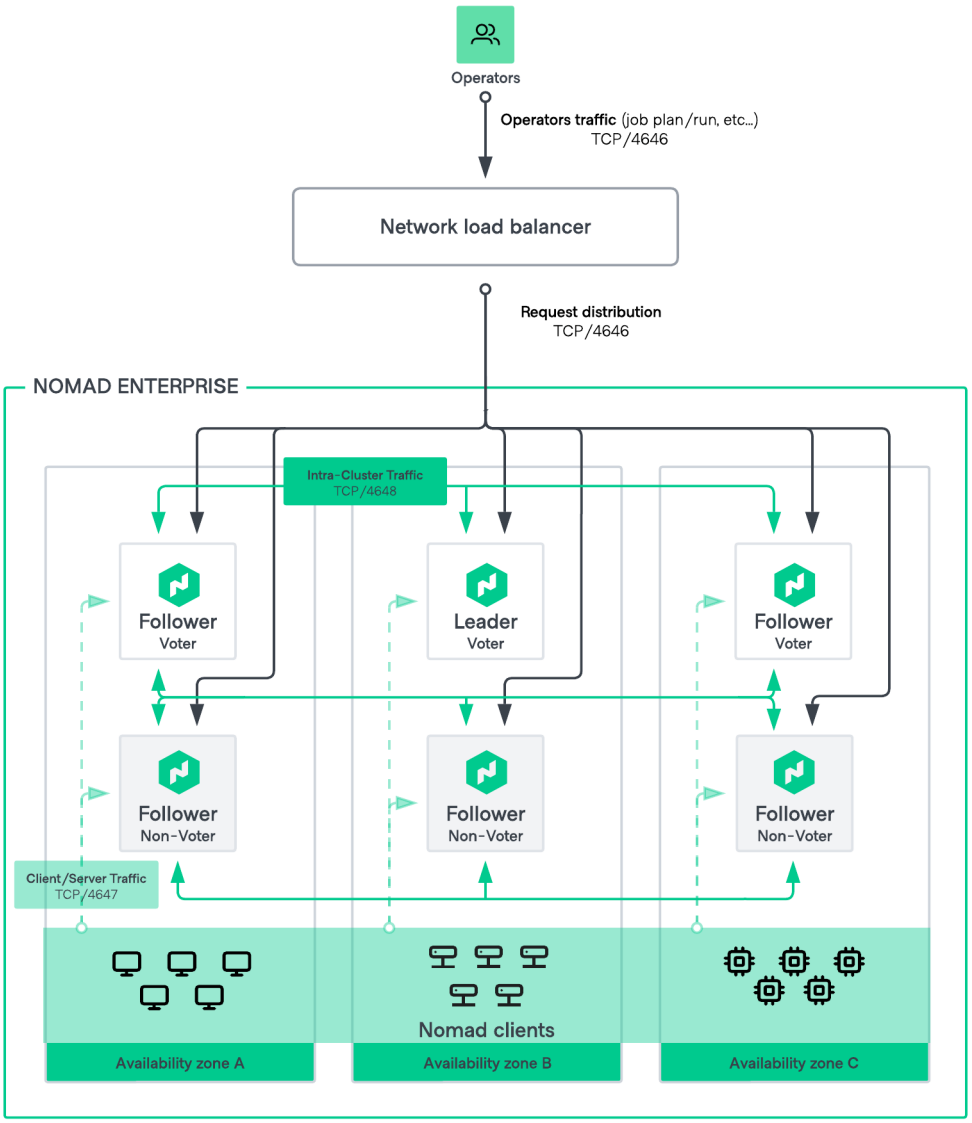


Figure 5: HVD Nomad networking diagram

The following table outlines the minimum network connectivity requirements for Nomad cluster nodes. You may also need to grant the Nomad servers outbound access to additional services that live elsewhere, either within your own internal network or via the internet.

Examples may include:

- authentication with SSO(OIDC)
- NTP, for maintaining consistent time between nodes
- remote log handlers, such as a Splunk or ELK environment
- metrics collection, such as Prometheus; etc.

Source	Destination	port	protoco	Direction	Purpose
Operators / administration	Load balancer	4646	tcp	incoming	Request distribution
Load balancer	Nomad servers	4646	tcp	incoming	Nomad API
Nomad clients	Nomad servers	4647	tcp	incoming	RPC communication between clients and servers
Nomad servers	Nomad servers	4648	tcp/udp	bidirectional	Cluster bootstrapping, gossip, replication, request forwarding
Nomad servers	External systems	variou	various	various	External APIs

Nomad generally should not be exposed to inbound traffic via a publicly-accessible endpoint. This is to minimize both the risk of exposing the API as well as the risk of denial-of-service attacks.

Wherever possible, access to the Nomad API should be limited to the networks used by Nomad operators.

> **Note**
>
> The amount of network bandwidth used by Nomad will depend entirely on the specific customer's usage patterns. The number of Nomad clients, jobs, healthchecks, and the frequency of job updates are all affecting network bandwidth. It's also important to consider bandwidth requirements to other external systems, such as monitoring and logging collectors.

4.2.3 Traffic Encryption

Every segment of Nomad-related network traffic should be encrypted in transit using TLS over HTTPS. This includes communication from clients to the Nomad servers, as well as communication

between operators and Nomad control plane (servers).

You will need to create a standard X.509 public certificate from an existing trusted certificate authority internal to your organization. This certificate will represent the Nomad cluster, and will be served by each Nomad server in the cluster. Along with the certificate itself, you will also need to supply the certificate's private key, as well as a bundle file from the certificate authority used to generate the certificate. The whole process is covered in the Enable TLS for Nomad tutorial.

You should also encrypt Nomad's server-to-server communication, using gossip encryption as it is described in the tutorial.

4.2.4 DNS

Internally, Nomad servers do not use DNS names to talk to other members of the cluster or to the clients, relying instead on IP addresses. However, you will need to configure a DNS record that resolves to the IP address hosting your load balancer configuration. This also will be the domain you will provision your TLS certificate for.

4.2.5 Load balancing

Use a TCP-based (layer 4) load balancer to distribute incoming requests across multiple Nomad servers within a cluster. Configure the load balancer to listen on port `4646/tcp` and forward requests to all Nomad servers in the cluster on port `4646/tcp`. *All* healthy nodes in the cluster should be able to receive requests in this way, including standby nodes (i.e., followers) and nodes configured as non-voters.

> **Tip**
>
> While this guide uses the default port 4646/tcp for the load balancer, using another port (such as 443/tcp) is also supported, and is a common practice for Nomad deployments.

4.2.6 TLS passthrough

Configure the load balancer to use TLS passthrough, rather than terminating the TLS connection at the load balancer. This keeps the traffic end-to-end encrypted from the operators to the Nomad servers, removing even the load balancer as a possible attack vector.

4.2.7 Health check

Each Nomad server node provides a health check API endpoint at `v1/agent/health`. HTTPS-based health checks provide a deeper check than simply checking the TCP port.

Configure your load balancer to perform a health check to all Nomad server nodes in a cluster by using the path `v1/agent/health`, with an expected status code of `200`. This will ensure that the Nomad servers are healthy and able to receive requests.

> Tip
>
> If possible, use a load balancer that supports both TLS passthrough and HTTPS-based health checks. AWS NLB, F5 BigIP, Citrix ADC, and HAProxy are just a few examples of load balancers that support this combination of functionality.Some load balancers, such as Nginx, do not allow for both TLS passthrough and the use of HTTPS-based health checks. If another type of load balancer cannot be used in your environment, you can still use a simple port-based check instead. Please note that port-based health checks are a more limited type of check, and may not detect issues that an HTTPS-based check would.

4.3 Software

To deploy production Nomad, this Validated Design only requires the Nomad Enterprise software binaries to be installed on the Nomad servers and clients.

The Nomad server process should not be run as root.

Instead, a dedicated user should be created and used both to run the Nomad service as well as to protect the files used by and created by Nomad.

4.4 Nomad License

Your Nomad Enterprise license must be deployed on every Nomad server node in order for the Nomad process to start.

4.5 Configuration

Nomad configuration file is referenced each time the Nomad process is started.

The configuration file is made up of a number of different stanzas. Each stanza is a section of the configuration file that configures a different aspect of Nomad's behavior. You can start running Nomad with minimal configuration thanks to its safe defaults. However, depending on your specific environment, you may need to fine-tune the settings.

4.6 Operating systems

The Validated Design Terraform module uses Ubuntu 22.04, but the same principles would apply to any other modern Linux distro. There are also Nomad releases for Windows and macOS.

4.7 Node Pools

Using node pools in HashiCorp Nomad offers several advantages:

1. **Isolation**: Node pools allow you to isolate workloads by grouping nodes with similar characteristics or purposes. This can be useful for separating production and development environments, or for isolating workloads with different security or compliance requirements.
2. **Resource Management**: By grouping nodes into pools, you can better manage and allocate resources, especially specific ones such as GPUs. This ensures that specific workloads have access to the resources they need without interference from other jobs.
3. **Affinity and Constraints**: While affinities and constraints can express preferences for certain nodes, node pools provide a more robust way to ensure that allocations for a job are placed on the desired set of nodes. This helps in maintaining consistency and predictability in job placement.
4. **Operational Efficiency**: Node pools simplify the management of large clusters by allowing you to apply configurations and policies at the pool level rather than individually to each node. This reduces operational overhead and complexity.
5. **Scalability**: As your infrastructure grows, node pools make it easier to scale by adding or removing nodes from specific pools without disrupting the overall cluster.
6. **Improved Scheduling**: Node pools enhance the scheduler's ability to make informed decisions about where to place workloads, leading to better utilization of resources and improved performance.

4.7.1 Example Use Cases:

- **Environment Separation**: Use separate node pools for production, staging, and development environments to ensure isolation and prevent resource contention.
- **Specialized Hardware**: Create node pools for nodes with specialized hardware, such as GPUs, to ensure that only jobs requiring those resources are scheduled on those nodes.
- **Compliance and Security**: Use node pools to group nodes that meet specific compliance or security requirements, ensuring that sensitive workloads are only placed on compliant nodes.

By leveraging node pools, you can achieve greater control, efficiency, and flexibility in managing your Nomad cluster.

4.8 Multi-region federation

You should consider using federation with HashiCorp Nomad in the following scenarios:

1. **Multi-Region Deployments**: If your infrastructure spans multiple geographic regions, federation can help manage and orchestrate workloads across these regions, ensuring high availability and disaster recovery.
2. **Workload Segmentation**: When you need to segment workloads for different business units, teams, or environments (e.g., production, staging, development), federation allow you to manage these segments independently while still maintaining a unified control plane.
3. **Resource Optimization**: federation enable you to optimize resource utilization by sharing workloads across multiple datacenters or cloud providers, balancing the load and reducing costs.
4. **Compliance and Security**: If you have specific compliance or security requirements that necessitate isolating certain workloads or data, federation can help enforce these policies while still providing centralized management.
5. **Scalability**: As your organization grows, federation allow you to scale your Nomad infrastructure horizontally by adding new regions or datacenters without disrupting existing workloads.
6. **Disaster Recovery**: federation provide a robust disaster recovery strategy by allowing you to failover workloads to another region or datacenter in case of an outage. Caveats apply, consult the related Federation architecture, operations, and Failure scenarios documents that cover the topic in detail.

4.8.1 Example Use Cases:

- **Global Applications**: Deploy applications that need to be available globally, ensuring low latency and high availability by distributing workloads across multiple regions.
- **Hybrid Cloud**: Manage workloads across on-premises datacenters and multiple cloud providers, optimizing for cost and performance.
- **Regulatory Compliance**: Isolate workloads that handle sensitive data to specific regions or datacenters to comply with local regulations.

By leveraging federation, you can achieve greater resilience, flexibility, and efficiency in managing your Nomad infrastructure.

5 Cloud deployment (AWS, Azure, GCP virtual machines)

This section provides a detailed guide for manually installing a Nomad cluster on AWS, Azure, and GCP instances. This approach is suitable for organizations that require fine-grained control over their Nomad deployment or have specific compliance requirements that necessitate manual installation. Before proceeding, ensure you have read and have a solid understanding of Nomad architecture and Nomad deployment requirements.

5.1 Architectural summary

As referenced in the Nomad architecture section each Nomad node in a cluster resides on a separate VM, with shared or dedicated tenancy both being acceptable. A single Nomad cluster deploys within a single region, with nodes distributed across all available availability zones (AZs). This design uses Nomad's redundancy zone feature and deploys a minimum of 3 voting server nodes and 3 non-voting server nodes across 3 AZs.

5.1.1 Server naming and IP addressing

For illustrative purposes, we use the following naming convention and IP addressing scheme for our Nomad servers:

DNS name	IP address	Node type	Location
nomad-1.domain	10.0.0.1	Voting server	us-east-1a
nomad-2.domain	10.0.0.2	Voting server	us-east-1b
nomad-3.domain	10.0.0.3	Voting server	us-east-1c
nomad-4.domain	10.0.0.4	Non-voting server	us-east-1a
nomad-5.domain	10.0.0.5	Non-voting server	us-east-1b
nomad-6.domain	10.0.0.6	Non-voting server	us-east-1c
nomad.domain	10.0.0.100	NLB	(all zones)

Note: These names and IP addresses are examples. Adjust them according to your specific environment and naming conventions.

5.1.2 Certificates

Create a standard X.509 certificate for installation on the Nomad servers. Follow your organization's process for creating a new certificate that matches the DNS record you intend to use for accessing Nomad. This guide assumes the conventions and uses self-signed certificates from the Nomad architecture and enable TLS tutorial.

You need three files:

1. CA Public certificate (`nomad-agent-ca.pem`)
2. Server node certificate's private key: (`global-server-nomad-key.pem`)
3. Server node public certificate: (`global-server-nomad.pem`)

Distribute these certificates to the server nodes in a later section and used to secure communications between Nomad clients, servers, and the API.

A fourth certificate and key is also needed: `global-cli-nomad.pem` and `global-cli-nomad-key.pem`. Use these later in this guide to interact with the cluster. The `global-` prefix for the certificate and key assume these are for multi-region environments. You are able to generate certificates on a per-region basis, however In this guide, we are using a single certificate and key across all clusters globally.

Note

Ensure the certificate's Common Name (CN) or Subject Alternative Names (SANs) include the DNS names you'll use to access Nomad, such as `nomad.yourdomain.com`. If you're using separate DNS names for server-to-server communication, include these in the SANs as well.

While Nomad's TLS configuration is production ready, key management and rotation is a complex subject not covered by this guide. Vault is the suggested solution for key generation and management.

5.1.3 Firewall rules

Create firewall rules that allow these ports bidirectionally:

Port	Name	Purpose	Protocol
464	HTTP API	Used by clients and servers to serve the HTTP API	TCP only
4647	RPC	Used for internal RPC communication between client agents and servers, and for inter-server traffic	TCP only
464	Serf WAN	Used by servers to gossip both over the LAN and WAN to other servers. Not required for Nomad clients to reach this address	TCP and UDP

5.1.4 Load balancing the API and UI

- Ensure your load balancer is in front of all Nomad server nodes, across all availability zones. Ensure you have deployed all Nomad nodes in the correct subnets.
- Use TCP ports 4646 (HTTP API) and 4647 (RPC). The target groups associated with each of these listeners must contain all Nomad server nodes.
- Use the `/v1/agent/health` endpoint for health checking.

5.1.5 Download and install the Nomad CLI

To interact with your Nomad cluster, you need to install the Nomad binary. Follow the steps for your OS on the Install Nomad documentation page.

5.1.6 Obtain the Nomad Enterprise license file

Obtain the Nomad Enterprise license file from your HashiCorp account team. This file contains a license key unique to your environment. The file is named `license.hclic` or similar.

Keep this file available as you need it later in the installation process.

5.2 Platform-specific guidance

Select the tab below for further guidance and corresponding choices for your cloud service provider.

5.2.1 Deployment considerations

AWS

The official Nomad Enterprise HVD Module for deployment on AWS is available in the Terraform Registry.For Nomad servers, we recommend using `m5.xlarge` or `m5.2xlarge` instance types to provide the necessary vCPU and RAM resources for Nomad. Instance types from other instance families are also usable, provided that they meet or exceed the resource recommendations.When selecting instance types for Nomad clients, consider the resource requirements of your workloads. AWS offers a variety of

5.3 Deployment process

5.3.1 Create and connect to your instances

1. Launch two instances in each of the three AZs, for a total of six instances using the machine image of your choice. Ubuntu is the OS used in the following example. Adjust these to use `dnf` on RHEL if needed.
2. Ensure the IAM Instance Profile or Managed Service Identity is using the Cloud auto-join role permissions you previously created.
3. Ensure the instances are tagged with `tag_key=cloud-auto-join` & `tag_value=true`
4. Update the OS

```
ssh -i /path/to/your-private-key ubuntu@your-vm-ip
sudo apt update && sudo apt upgrade -y
```

5.3.2 Download and install Nomad

Refer to the HashiCorp Security page regarding secure download and verification of the Nomad single binary.

```
PRODUCT='nomad'
VERSION='1.9.1+ent'
OS_ARCH="$(uname -s | tr '[:upper:]' '[:lower:]')_$(uname -m | tr '[:upper:]' '[:
    lower:]')" # e.g. linux_x86_64
hashicorp_key_id='34365D9472D7468F'
hashicorp_key_fingerprint='C874011F0AB405110D02105534365D9472D7468F'
curl -#Ok --remote-name https://www.hashicorp.com/.well-known/pgp-key.txt
gpg --import pgp-key.txt
gpg --sign-key ${hashicorp_key_id}
gpg --fingerprint --list-signatures "HashiCorp Security" | tr -d ' ' | grep -q "${
    hashicorp_key_fingerprint}"
curl -#Ok --remote-name https://releases.hashicorp.com/"${PRODUCT}"/"${VERSION}"/"${
    PRODUCT}"_"${VERSION}"_"${OS_ARCH}".zip
curl -#Ok --remote-name https://releases.hashicorp.com/"${PRODUCT}"/"${VERSION}"/"${
    PRODUCT}"_"${VERSION}"_SHA256SUMS
curl -#Ok --remote-name https://releases.hashicorp.com/"${PRODUCT}"/"${VERSION}"/"${
    PRODUCT}"_"${VERSION}"_SHA256SUMS.sig
gpg --verify "${PRODUCT}"_"${VERSION}"_SHA256SUMS.sig "${PRODUCT}"_"${VERSION}"
    _SHA256SUMS
[[ "$(sha256sum ${PRODUCT}_${VERSION}_${OS_ARCH}.zip | awk '{print $1}')" != "$(grep
    ${PRODUCT}_${VERSION}_${OS_ARCH}.zip "${PRODUCT}"_"${VERSION}"_SHA256SUMS | awk
    '{print $1}')" ]] && echo "ALERT: VERIFICATION FAILED" || echo "DOWNLOAD
    VERIFICATION SUCCEEDED - PLEASE PROCEED TO UNZIP AND MOVE INTO PLACE"  # stop if
    verification fails
unzip "${PRODUCT}"_"${VERSION}"_"${OS_ARCH}".zip
sudo mv "${PRODUCT}" /usr/local/bin/
sudo chmod -h 0755 /usr/local/bin/nomad
```

Follow the instructions in the documentation page to install Nomad from the official HashiCorp package repository for your OS.

Create necessary directories

```
sudo mkdir -p /opt/nomad/data
sudo mkdir -p /etc/nomad.d
sudo mkdir -p /etc/nomad.d/tls
```

Copy TLS certificates

Copy your TLS certificates to `/etc/nomad.d/tls/`:

```
sudo scp nomad-agent-ca.pem global-server-nomad-key.pem global-server-nomad.pem \
     ubuntu@your-vm-ip:/etc/nomad.d/tls/
```

Copy license file

```
echo "02MV4UU43BK5..." >> /etc/nomad.d/license.hclic
```

Create gossip encryption key

```
nomad operator gossip keyring generate
```

Save the output of this key to use in the configuration file.

> Note
>
> The `nomad operator gossip keyring generate` command returns 16 bytes; however, Nomad supports gossip encryption keys of 32 bytes as well. Supplying your own 32 byte key enables AES-256 mode, where supplying a 16 byte key enables AES-128 mode.

Create Nomad configuration file

Create `/etc/nomad.d/nomad.hcl` with the following content or supply your own. The example below includes:

- Cloud Auto-join which joins servers to the cluster based on tags instead of hard coded IPs or hostnames.
- Autopilot which handles automated upgrades
- Redundancy zones: Nomad uses these values to partition the servers by redundancy zone, and keeps one voting server per zone. Extra servers in each zone stay as non-voters on standby and are promoted automatically if the active voter in that availability zone goes offline. We recommend matching the name of the AWS availability zone for simplicity. Change your config based on which AZ the instance resides in.
- TLS: enables TLS between nodes and for the API. At scale, it is recommended to use HashiCorp Vault, however for starting out, it is acceptable to leverage signed certificates copied to the instance.

AWS

```
data_dir = "/opt/nomad/data"
bind_addr = "0.0.0.0" # CHANGE ME to listen only on the required IPs

server {
  enabled = true
  bootstrap_expect = 3
  server_join {
    retry_join = ["provider=aws tag_key=cloud-auto-join tag_value=true"]
  }
  redundancy_zone = "us-east-1b" # CHANGE ME
  license_path = "/etc/nomad.d/license.hclic"
  encrypt = "YOUR-GOSSIP-KEY" # Paste your key from the above "Create Gossip
      Encryption Key" step
}

acl {
  enabled = true
}

tls {
  http = true
  rpc  = true
  ca_file   = "/etc/nomad.d/tls/nomad-agent-ca.pem"
  cert_file = "/etc/nomad.d/tls/global-server-nomad-key.pem"
  key_file  = "/etc/nomad.d/tls/global-server-nomad.pem"
}

autopilot {
  cleanup_dead_servers      = true
  last_contact_threshold    = "200ms"
  max_trailing_logs         = 250
  server_stabilization_time = "10s"
  enable_redundancy_zones   = true
  disable_upgrade_migration = false
  enable_custom_upgrades    = false
}
```

Azure

```
data_dir = "/opt/nomad/data"
bind_addr = "0.0.0.0" # CHANGE ME to listen only on the required IPs

server {
  enabled = true
  bootstrap_expect = 3
  server_join {
    retry_join = ["provider=azure tag_name=... tag_value=... tenant_id=...
      client_id=... subscription_id=... secret_access_key=..."]
  }
  redundancy_zone = "us-east-1b" # CHANGE ME
  license_path = "/etc/nomad.d/license.hclic"
  encrypt = "YOUR-GOSSIP-KEY" # Paste your key from the above "Create Gossip
      Encryption Key" step
```

If you are using a non-default scheduler configuration, ensure these settings are set before starting Nomad for the first time, or subsequently set via the API.

5.3.3 Starting Nomad

Set up Nomad service

```
sudo nano /etc/systemd/system/nomad.service
```

```
[Unit]
Description=Nomad
Documentation=https://www.nomadproject.io/docs/
Wants=network-online.target
After=network-online.target

[Service]

# Nomad servers should be run as the nomad user.
# Nomad clients should be run as root
User=root
Group=root

ExecReload=/bin/kill -HUP $MAINPID
ExecStart=/usr/local/bin/nomad agent -config /etc/nomad.d
KillMode=process
KillSignal=SIGINT
LimitNOFILE=65536
LimitNPROC=infinity
Restart=on-failure
RestartSec=2

TasksMax=infinity
OOMScoreAdjust=-1000

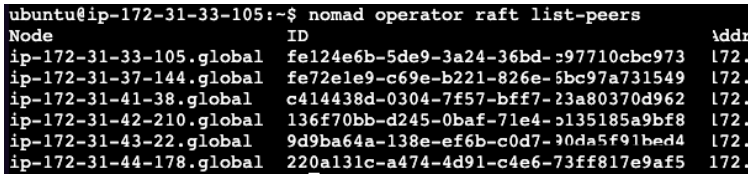
[Install]
WantedBy=multi-user.target
```

Start the Nomad service

```
sudo systemctl enable nomad
sudo systemctl start nomad
```

5.3.4 Validate the cluster is bootstrapped and functioning

Ensure you have 3 of the members state `Voter=False`:



```
ubuntu@ip-172-31-33-105:~$ nomad operator raft list-peers
Node                          ID                                    Addr
ip-172-31-33-105.global       fe124e6b-5de9-3a24-36bd-397710cbc973  172.
ip-172-31-37-144.global       fe72e1e9-c69e-b221-826e-5bc97a731549  172.
ip-172-31-41-38.global        c414438d-0304-7f57-bff7-23a80370d962  172.
ip-172-31-42-210.global       136f70bb-d245-0baf-71e4-b135185a9bf8  172.
ip-172-31-43-22.global        9d9ba64a-138e-ef6b-c0d7-30da5f91bed4  172.
ip-172-31-44-178.global       220a131c-a474-4d91-c4e6-73ff817e9af5  172.
```

If you are running into issues with your cluster bootstrapping or the Nomad agent starting, you can view journal logs to reveal any errors:

```
sudo systemctl status nomad
sudo journalctl -xe
```

Typically issues are due to invalid configuration entries within `nomad.hcl`, certificate, or networking issues where the nodes cannot communicate with each other.

5.3.5 Initialize ACL system

To initialize the Access Control List (ACL) system, you need to bootstrap it. This step is crucial for setting up access control policies in your Nomad cluster. The bootstrap process generates an initial management token that allows you to configure ACLs.

Log into one of your servers and run the following commands:

Set environment variables

```
export NOMAD_ADDR=https://127.0.0.1:4646
export NOMAD_CACERT=/etc/nomad.d/tls/nomad-agent-ca.pem
export NOMAD_CLIENT_CERT=/etc/nomad.d/tls/global-server-nomad.pem
export NOMAD_CLIENT_KEY=/etc/nomad.d/tls/global-server-nomad-key.pem
```

Bootstrap the ACL system. This only needs to be done on one server. The ACLs sync across server nodes.

```
nomad acl bootstrap
```

Save the accessor ID and secret ID and follow the Identity and Access Management section of the Nomad Administration Guide for more information on ACLs and post installation tasks.

5.3.6 Interact with your Nomad cluster

```
export NOMAD_ADDR=https://nomad-load-balancer
export NOMAD_TOKEN=YourSecretID
export NOMAD_CACERT=./nomad-agent-ca.pem
export NOMAD_CLIENT_CERT=./global-cli-nomad.pem
export NOMAD_CLIENT_KEY=./global-cli-nomad-key.pem
nomad -autocomplete-install && complete -C /usr/bin/nomad nomad
```

5.3.7 Nomad clients

This section provides guidance on deploying and managing Nomad clients. Nomad clients are responsible for running tasks and jobs scheduled by the Nomad servers. They register with the server cluster, receive work assignments, and execute tasks.

Preparation

Many of the preparation steps for Nomad clients are similar to those for servers. Refer to the Platform-specific guidance section for details on the following:

- Creating security groups
- Setting up IAM roles
- Preparing for launching instances

Nomad client instance types

Organize these instance families within Nomad using node pools, allowing for workload segregation and enabling job authors to target their workload to the most appropriate hardware resources.

Deployment process

Follow the steps in the preparation and installation process sections of the server deployment section, with a few key differences in configuration. Spread your clients across availability zones.

Client TLS configuration

Use a different certificate for Nomad clients than the server nodes. From the same directory as the CA certificate you generated during the server installation steps:

1. `nomad tls cert create -client`

   This generates the following.

   - `global-client-nomad.pem` (client certificate)
   - `global-client-nomad-key.pem` (client private key)

2. Copy certificates to the client instances:

```
scp nomad-agent-ca.pem global-client-nomad.pem global-client-nomad-key.pem \
    ubuntu@your-vm-ip:/etc/nomad.d/tls/
```

Remember to keep your private keys secure and implement proper certificate management practices, including regular rotation and secure distribution of certificates to your Nomad clients.

In your `nomad.hcl` file:

AWS

```
data_dir = "/opt/nomad/data"
bind_addr = "0.0.0.0" #CHANGE ME to listen only on the required IPs

client {
  enabled = true
  server_join {
    retry_join = ["provider=aws tag_key=cloud-auto-join tag_value=true"]
  }
}

tls {
  http = true
  rpc = true
  ca_file = "/etc/nomad.d/tls/nomad-agent-ca.pem"
  cert_file = "/etc/nomad.d/tls/global-client-nomad.pem"
  key_file = "/etc/nomad.d/tls/global-client-nomad-key.pem"
}

#...other client configurations...
```

Azure

```
data_dir = "/opt/nomad/data"
bind_addr = "0.0.0.0" #CHANGE ME to listen only on the required IPs

client {
  enabled = true
  server_join {
    retry_join = ["provider=azure tag_name=... tag_value=... tenant_id=...
        client_id=... subscription_id=... secret_access_key=..."]
  }
}

tls {
  http = true
  rpc = true
  ca_file = "/etc/nomad.d/tls/nomad-agent-ca.pem"
  cert_file = "/etc/nomad.d/tls/global-client-nomad.pem"
  key_file = "/etc/nomad.d/tls/global-client-nomad-key.pem"
}

#...other client configurations...
```

GCP

```
data_dir = "/opt/nomad/data"
bind_addr = "0.0.0.0" #CHANGE ME to listen only on the required IPs

client {
  enabled = true
  server_join {
    retry_join = ["provider=gce project_id=... tag_value=... zone_pattern=..."
```

5.3.8 Post-installation tasks

After installation, perform these tasks to ensure everything is working as expected:

Verify cluster health:

```
nomad server members #lists all servers
nomad node status #lists all clients
```

For day two activities such as observability and backups, consult the Nomad Administration Guide.

6 Manual installation of Nomad Enterprise

This guide provides detailed instructions for deploying Nomad Enterprise in on-premises environments, covering both virtual machine and bare metal installations. Before proceeding, ensure you have reviewed the Nomad Architecture and Deployment Requirements.

6.1 Preparation

6.1.1 Server requirements

For Nomad servers, hardware requirements depends on the size of the cluster and the rate of change. As a rule of thumb, we recommend the following specifications based on cluster size.

Cluster Size	CPU Cores	Memory	Storage
Small (< 100 clients)	4-8 cores	16-32GB	50GB SSD
Medium (100-500 clients)	8-16 cores	32-64GB	100GB SSD
Large (500+ clients)	16-32 cores	64-128GB	200GB+ SSD

Nomad servers can be IOPS intensive as it writes chunks of data from memory to storage. Ensure that the storage subsystem can handle the write load (SSDs are a must), and be sure to monitor disk latency and throughput.

For the continued availability of the Nomad cluster, use 5-6 servers in the cluster. Nomad uses the Raft consensus algorithm, so ensure that there is always a quorum. The best way to achieve this depends a bit on the available network and datacenter topology.

- If there are more than 3 failure domains (datacenters close to one another (< 10ms latency), independent datacenter rooms or racks, etc), deploy 6 servers and make use of the Redundancy Zones feature, which ensures that there are the correct number of voting servers in each failure domain, and increases the fault tolerance and performance of the cluster.
- If there only 2 failure domains (two datacenters), where the system cannot maintain quorum, consider either deploying the servers in only one of the failure domains, or deploying two clusters of 5 servers each, federated together.
- If there are not any clear failure domains (single datacenter), deploy 5 servers.

6.1.2 Client requirements

Client hardware requirements vary based on workload types. Generally, unless you are running in an *edge* scenario, avoid having small client sizes. As a rule of thumb, aim for at least 4 cores and 8GB of memory per client, adapting as needed based on workload requirements and distribution. Connect Nomad clients to the servers in any datacenter, even those connected over unreliable networks (in which case, tune the behavior of workloads using the disconnect block).

Nomad clients can run on virtual machines or bare metal servers, depending on your organization's requirements. Start by running clients in virtual machines, and as the number of workloads increases, consider moving to bare metal for better performance, resource utilization and to avoid having two orchestrators trying to optimize the use of the same underlying hardware.

Consider reserving resources on the hypervisor level for VM-based Nomad clients to ensure available physical capacity for assigned workloads.

6.1.3 Network requirements

Nomad requires a few ports for internal communication, bidirectionally. The defaults for those are:

Port	Name	Purpose	Protocol
4646	HTTP API	Used by clients and servers to serve the HTTP API	TCP only
4647	RPC	Used for internal RPC communication between client agents and servers, and for inter-server traffic	TCP only
4648	Serf WAN	Used by servers to gossip both over the LAN and WAN to other servers. Not required for Nomad clients to reach this address	TCP and UDP

For optimal performance, ensure the following.

- Low-latency connectivity between servers (< 10 ms round-trip time)
- Sufficient bandwidth for job artifacts and container images
- Network segmentation between server and client traffic

6.1.4 Certificates

Create a standard X.509 certificate for the Nomad servers. Follow your organization's process for creating a new certificate that matches the DNS record you intend to use for accessing Nomad. This guide assumes the conventions and using self-signed certificates from the Nomad Architecture and Enable TLS tutorial.

You need three files.

1. CA Public certificate (`nomad-agent-ca.pem`)
2. Server node certificate's private key: (`global-server-nomad-key.pem`)
3. Server node public certificate: (`global-server-nomad.pem`)

Distribute these certificates to the server nodes (covered below). They are used to secure communications between Nomad clients, servers, and the API.

Generate a fourth certificate and private key used later in this guide for clients: `global-cli-nomad.pem` and `global-cli-nomad-key.pem`. The `global-` prefix for the certificate and key assume these are for multi-region environments. You are able to generate certificates on a per-region basis, however, in this guide, we are using a single certificate and key for use across all clusters globally.

> **Note**
>
> Ensure the certificate's Common Name (CN) or Subject Alternative Names (SANs) include the DNS names you will use to access Nomad, such as `nomad.yourdomain.com`. If you are using separate DNS names for server-to-server communication, include these in the SANs as well.

While the TLS configuration included in this guide provides a production ready configuration, key management and rotation is a complex subject not covered by this guide. Vault PKI integration with Nomad is the suggested solution for key generation and management.

6.1.5 Load balancing the API and UI

Load balance the UI and API of the servers (either for human/machine access, CI/CD pipeline consumption, or for initial discovery to join the cluster), you can use any load balancer, in layer 4 or layer 7 mode (caveat: the web UI's `exec` into an allocation uses web sockets, so if you need it for debugging purposes, your load balancer must allow for that). Use the `/v1/agent/health` endpoint for health checking. Terminate TLS at the load balancer to enable full mTLS with client verification without impacting human users connecting to the web UI.

6.1.6 Auto-join

Use the `server_join` configuration option to specify the initial set of Nomad servers to join (for both servers to form the cluster, and clients to join it). This can be a list of IP addresses or DNS names of the servers or a load balancer in front of them, or a cloud auto-join configuration. There are cloud auto-join providers for a number of on-premises orchestrators (such as VMware vSphere, OpenStack), as well as mDNS. At least one of the provided IPs/DNS names/cloud auto-join providers must be reached for the Nomad agent to join the cluster. Once this happens, all other servers are automatically discovered, so it is not an issue if some of the servers are unreachable.

Servers:

```
server {
  enabled          = true
  bootstrap_expect = 3
  server_join {
    retry_join     = [ "1.1.1.1", "2.2.2.2" ]
  }
}
```

Clients:

```
client {
    enabled = true
    server_join {
        retry_join     = [ "1.1.1.1", "2.2.2.2" ]
    }
}
```

6.1.7 Obtain the Nomad Enterprise license file

Obtain the Nomad Enterprise License File from your HashiCorp account team. This file contains a license key unique to your environment. Name the file `nomad.hclic`. Keep this file on hand, as it is needed later in the installation process.

6.1.8 Download and install the Nomad command-line tool

To interact with your Nomad cluster, install the Nomad binary. Follow the steps for your OS on the Install Nomad documentation page.

6.2 Deployment process

6.2.1 Infrastructure bootstrapping

Create the necessary infrastructure for your Nomad cluster. Use the most appropriate tooling such as bare metal machines via PXE boot, or an automated provisioning system such as Canonical MaaS or Tinkerbell. For virtual machines, use Terraform. Ensure that the servers have the necessary network connectivity and storage available, and set up any related infrastructure (such as load balancers, firewall, and switch configurations, DNS entries, etc.)

Automate the following steps using your preferred configuration management and image management tools ensuring consistency and repeatability.

6.2.2 Download and install Nomad

Manual

```
PRODUCT='nomad'
VERSION='1.9.1+ent'
OS_ARCH="$(uname -s | tr '[:upper:]' '[:lower:]')_$(uname -m | tr '[:upper:]'
    '[:lower:]')" # e.g. linux_x86_64
hashicorp_key_id='34365D9472D7468F'
hashicorp_key_fingerprint='C874011F0AB405110D02105534365D9472D7468F'
curl -#Ok --remote-name https://www.hashicorp.com/.well-known/pgp-key.txt
gpg --import pgp-key.txt
gpg --sign-key ${hashicorp_key_id}
gpg --fingerprint --list-signatures "HashiCorp Security" | tr -d ' ' | grep -q
     "${hashicorp_key_fingerprint}"
curl -#Ok --remote-name https://releases.hashicorp.com/"${PRODUCT}"/"${VERSION
    }"/"${PRODUCT}"_"${VERSION}"_"${OS_ARCH}".zip
curl -#Ok --remote-name https://releases.hashicorp.com/"${PRODUCT}"/"${VERSION
    }"/"${PRODUCT}"_"${VERSION}"_SHA256SUMS
curl -#Ok --remote-name https://releases.hashicorp.com/"${PRODUCT}"/"${VERSION
    }"/"${PRODUCT}"_"${VERSION}"_SHA256SUMS.sig
gpg --verify "${PRODUCT}"_"${VERSION}"_SHA256SUMS.sig "${PRODUCT}"_"${VERSION}
    "_SHA256SUMS
[[ "$(sha256sum ${PRODUCT}_${VERSION}_${OS_ARCH}.zip | awk '{print $1}')" != "
    $(grep ${PRODUCT}_${VERSION}_${OS_ARCH}.zip "${PRODUCT}"_"${VERSION}"
    _SHA256SUMS | awk '{print $1}')" ]] && echo "ALERT: VERIFICATION FAILED"
    || echo "DOWNLOAD VERIFICATION SUCCEEDED - PLEASE PROCEED TO UNZIP AND
    MOVE INTO PLACE"  # stop if verification fails
unzip "${PRODUCT}"_"${VERSION}"_"${OS_ARCH}".zip
sudo mv "${PRODUCT}" /usr/local/bin/
sudo chmod -h 0755 /usr/local/bin/nomad
```

Package repository

Follow the instructions in the documentation page to install Nomad from the official HashiCorp package repository for your OS.

6.2.3 Create necessary directories

```
sudo mkdir -p /opt/nomad/data
sudo mkdir -p /etc/nomad.d
sudo mkdir -p /etc/nomad.d/tls
```

6.2.4 Copy TLS certificates

Copy your TLS certificates to `/etc/nomad.d/tls/`.

```
sudo scp nomad-agent-ca.pem global-server-nomad-key.pem global-server-nomad.pem
    ubuntu@server-machine:/etc/nomad.d/tls/
```

6.2.5 Copy license file

```
echo "02MV4UU43BK5..." >> /etc/nomad.d/license.hclic
```

6.2.6 Create gossip encryption key

```
nomad operator gossip keyring generate
```

Save the output of this key to use in the configuration file.

> Note
>
> The `nomad operator gossip keyring generate` command returns 16 bytes. However, Nomad supports gossip encryption keys of 32 bytes as well. Supplying your own 32 byte key enables AES-256 mode, where supplying a 16 byte key enables AES-128 mode.

6.2.7 Create Nomad configuration file

Create `/etc/nomad.d/nomad.hcl` with the following content or supply your own. The example below includes:

- Autopilot which handles automated upgrades

- Redundancy zones: Nomad uses these values to partition the servers by redundancy zone/-failure domain, and aims to keep one voting server per zone. Extra servers in each zone stay as non-voters on standby and Nomad promotes these the respective active voter leaves or dies.
- TLS: enables TLS between nodes and for the API. At scale, we recommend using HashiCorp Vault, however for starting out, it is acceptable to use signed certificates copied to the machine.

```
data_dir = "/opt/nomad/data"
bind_addr = "0.0.0.0" #CHANGE ME to listen only on the required IPs

server {
  enabled = true
  bootstrap_expect = 3
  server_join {
    retry_join     = [ "1.1.1.1", "2.2.2.2" ] # CHANGE ME
  }
  redundancy_zone = "dc1-rack-b51" # CHANGE ME
  license_path = "/etc/nomad.d/license.hclic"
  encrypt = "YOUR-GOSSIP-KEY" #Paste your key from the above "Create Gossip
      Encryption Key" step
}

acl {
  enabled = true
}

tls {
  http = true
  rpc  = true
  ca_file   = "/etc/nomad.d/tls/nomad-agent-ca.pem"
  cert_file = "/etc/nomad.d/tls/global-server-nomad-key.pem"
  key_file  = "/etc/nomad.d/tls/global-server-nomad.pem"
}

autopilot {
  cleanup_dead_servers      = true
  last_contact_threshold    = "200ms"
  max_trailing_logs         = 250
  server_stabilization_time = "10s"
  enable_redundancy_zones   = true
  disable_upgrade_migration = false
  enable_custom_upgrades    = false
}
```

If you are using a non-default scheduler configuration, configure these settings and then start

Nomad for the first time. Not doing this requires subsequent configuration using the API.

6.2.8 Starting Nomad

Manual

Create the Nomad service file:“‘ sudo nano /etc/systemd/system/nomad.service ““‘ [Unit] Description=Nomad Documentation=https://www.nomadproject.io/docs/ Wants=network-online.target After=network-online.target
[Service]

7 Nomad servers should be run as the nomad user.

8 Nomad clients should be run as root

User=root Group=root
ExecReload=/bin/kill -HUP $MAINPID ExecStart=/usr/local/bin/nomad agent -config /etc/nomad.d KillMode=process KillSignal=SIGINT LimitNOFILE=65536 LimitNPROC=infinity Restart=on-failure RestartSec=2
TasksMax=infinity OOMScoreAdjust=-1000
[Install] WantedBy=multi-user.target `Enable and start the Nomad service`: sudo systemctl enable nomad sudo systemctl start nomad

```
:::

::: {data-environment=tcolorbox data-environment-keyword="colframe=
    HexColor656A76,coltitle=HexColor656A76,standard jigsaw,title=Package,
    opacityback=0.0,opacitybacktitle=0.0"}
Enable and start the Nomad service:```
sudo systemctl enable nomad
sudo systemctl start nomad
```

8.0.1 Validate the cluster

Verify you have the expected amount of servers with the following commands.

```
nomad server members
nomad operator raft list-peers
```

If you are running into issues with your cluster bootstrapping or the Nomad agent starting, you can view journal logs to reveal any errors:

```
sudo systemctl status nomad
sudo journalctl -xe
```

Typically issues are due to invalid configuration entries within `nomad.hcl`, certificate, or networking issues where the nodes cannot communicate with each other.

8.0.2 Initialize ACL system

Log into one of your servers and run the following commands:

Set environment variables

```
export NOMAD_ADDR=https://127.0.0.1:4646
export NOMAD_CACERT=/etc/nomad.d/tls/nomad-agent-ca.pem
export NOMAD_CLIENT_CERT=/etc/nomad.d/tls/global-server-nomad.pem
export NOMAD_CLIENT_KEY=/etc/nomad.d/tls/global-server-nomad-key.pem
```

Bootstrap the ACL system. Do this on only one of the servers in the cluster. Nomad syncs ACLs across all cluster nodes.

```
nomad acl bootstrap
```

Save the accessor ID and Secret ID and follow the Identity and Access Management section of the Nomad Administration Guide for more information on ACLs and post installation tasks.

8.0.3 Interact with your Nomad cluster

```
export NOMAD_ADDR=https://nomad-lb
export NOMAD_TOKEN=YourSecretID
export NOMAD_CACERT=./nomad-agent-ca.pem
export NOMAD_CLIENT_CERT=./global-cli-nomad.pem
export NOMAD_CLIENT_KEY=./global-cli-nomad-key.pem
nomad -autocomplete-install && complete -C /usr/bin/nomad nomad
```

8.0.4 Client installation

The client installation process is similar to the server installation, with a few key differences in the configuration. Nomad clients are responsible for running tasks and jobs scheduled by the Nomad servers. They register with the server cluster, receive work assignments, and execute tasks.

Client TLS configuration

Nomad clients must use a different certificate than the server nodes. From the same directory as the CA certificate you generated during the server installation steps:

1. `nomad tls cert create -client`

   This generates the following.

   - `global-client-nomad.pem` (Client certificate)
   - `global-client-nomad-key.pem` (Client private key)

2. Copy certificates to the client machines using this command.

```
scp nomad-agent-ca.pem global-client-nomad.pem global-client-nomad-key.pem
ubuntu@client-machine:/etc/nomad.d/tls/
```

Remember to keep your private keys secure and implement proper certificate management practices, including regular rotation and secure distribution of certificates to your Nomad clients.

In your `nomad.hcl` file:

```
# Base configuration
data_dir = "/opt/nomad/data"
bind_addr = "0.0.0.0" #CHANGE ME to listen only on the required IPs

# Client configuration
client {
  enabled = true

  # Server join configuration
    server_join {
        retry_join     = [ "1.1.1.1", "2.2.2.2" ]
    }

  # Node pool configuration
  # Optionally configure a node pool for this client to join
  # node_pool = "general"
}

# Security configuration
tls {
  http = true
  rpc  = true
  verify_server_hostname = true
  ca_file   = "/etc/nomad.d/tls/nomad-agent-ca.pem"
  cert_file = "/etc/nomad.d/tls/client.pem"
  key_file  = "/etc/nomad.d/tls/client-key.pem"
}

# Telemetry configuration
telemetry {
  publish_allocation_metrics = true
  publish_node_metrics = true
  prometheus_metrics = true
}
```

8.1 Post-installation tasks

After installation, perform these tasks to ensure everything worked as expected:

Verify cluster health:

```
nomad server members #lists all servers
nomad node status #lists all clients
```

For day 2 activities, such as observability and backups, consult the Nomad Administration Guide.

9 Windows clients

Nomad Enterprise's platform-agnostic design extends to Windows environments, including Windows Subsystem for Linux 2 (WSL2). While core functionalities remain consistent, operating Nomad Enterprise on Windows presents unique considerations in file systems, process management, networking, and security.

This section highlights these Windows-specific nuances, offering key insights and best practices for effectively implementing and managing Nomad Enterprise in Windows ecosystems.

9.1 Path conventions

Golang uses a backslash \ as an escape character which conflicts with Windows file system conventions. To work around this, utilize a double backslash for Windows specific paths.

```
# Windows data_dir = "C:\\nomad\\data"
# Linux/WSL2 data_dir = "/opt/nomad/data"
```

9.2 Allocation isolation

Windows does not have a concept of Cgroups, therefore there is no filesystem isolation that occurs within Windows which means the `exec` and `exec2` drivers are not supported. For any executable workloads, you **must** use the raw_exec driver. This creates a security concern were the allocation ever compromised, as the entire underlying Windows host is then exposed.

> Note
>
> There are some design considerations to work around this. Refer to Installation Guide (Self-Managed) for design considerations.

9.3 Docker and Podman orchestration

It is important to note that the correct setup and configuration of Docker and Podman on Windows are prerequisites for Nomad Enterprise's integration. However, the specifics of configuring these container engines are beyond the scope of this document.

For detailed guidance on setting up Docker and Podman on Windows, please refer to their respective official documentation:

- Docker for Windows
- Podman on Windows

For information on configuring Nomad to work with Docker and Podman, please consult the official Nomad documentation:

- Nomad Docker driver
- Nomad Podman driver

9.3.1 Allowed caps

You need to define `allow_caps=["all"]` in your Docker plugin configuration, otherwise container workloads fail to deploy.

9.4 IPv6

Unless you are using IPv6, Nomad Enterprise advertises the ipv6 address by default over IPv4. Disable this functionality by setting the `advertise_ipv6_address = false` in the Nomad client configuration or completely turn off IPv6 on the network interface.

9.5 Logs

Nomad Enterprise logging on Windows is not automatically integrated with the Windows Event Viewer. By default, these log files continuously grow without any built-in size limitation, potentially consuming all available disk space. To prevent this, it is highly recommended to implement log rotation. You can configure Nomad Enterprise to limit the total log storage to 5MB using the following log rotation options:

```
client {
    log_file = "C:\\Nomad\\logs.txt"
    log_rotate_bytes = "100000"
    log_rotate_max_files = "5"
}
```

9.6 Volumes

SMB mounts are commonly used in Windows environments for various workloads. Nomad Enterprise can leverage these existing drive mounts, however for container workloads, the underlying Windows filesystem is not accessible. To enable this, add the following to your Nomad client configuration.

```
plugin "docker" {
    config {
        volumes {
            enabled = true
        }
  }
}
```

This configuration enables Nomad Enterprise to access all Windows volumes on the host, including but not limited to SMB mounts. It grants containerized workloads the ability to utilize any mounted drive or volume present on the Windows host system.

9.7 WSL2 networking

By default, WSL uses a NAT (network address translation) based architecture for networking. This is not ideal when accessing WSL2-hosted applications. Enable network mirror mode to enable the newer networking architecture for WSL2 and allow external access to your applications within WSL2.

9.8 Example Nomad Windows client configuration

```
data_dir = "C:\\Nomad"

# Specifies which address the Nomad agent should bind to for network services,
# including the HTTP interface as well as the internal gossip protocol and RPC
    mechanism.

bind_addr = "0.0.0.0"

log_file = "C:\\Nomad\\logs.txt"
log_rotate_bytes = "100000"
log_rotate_max_files = "5"

client {
    enabled = true

#Specifies a list of server addresses to join and will continue to attempt
#to join even after a failure at a specified occurrence.
    server_join {
        retry_join = [ "1.1.1.1", "2.2.2.2", "3.3.3.3" ] #Replace these with the IPs
            of the server nodes
        retry_max = 3
        retry_interval = "15s"
    }
}

plugin "docker" {
    allow_caps = ["all"]
    config {
        volumes {
            enabled = true
        }
    }
}
```

10 Post-installation configuration

> **Note**
>
> This page covers one-time first-run bootstrap tasks after deployment. For ongoing operator governance and configuration-as-code workflows, see the Administration Guide: Configuration as code.

After deploying and bootstrapping Nomad Enterprise, follow these steps to configure the servers.

1. Enable mTLS and gossip encryption
2. Enable and configure access control lists and OIDC for RBAC
3. Set up automated upgrades to ensure Nomad cluster stays up-to-date with the latest features and security patches
4. Implement a robust backup strategy to protect your Nomad cluster's state and configuration
5. Load balance and reverse proxy for Nomad's web UI
6. Use Terraform to manage Nomad configuration as code, enabling version control and reproducibility

11 Next steps

After completing the Nomad Enterprise installation, continue with the sibling HVD guides:

- Use the Administration Guide to configure ongoing platform operations, access control, upgrades, backups, observability, policy, quotas, and scaling.
- Use the User Guide to help application teams deploy, scale, network, discover, and secure workloads on the platform.

12 Changelog

12.1 2026-08

- Restructured legacy Nomad HVD content into the HVD 2.0 Nomad Installation Guide as part of the Installation / Administration / User guide migration.

12.2 2026-01

- Fixed heading formatting in tabbed content for PDF generation
- Update meta descriptions
- Updated credits and contributors

12.3 2025-10

- Nomad sdg

12.4 2025-07

- Updated documentation links

12.5 2025-06

- Update installation page
- Init Nomad GCP SDG

12.6 2025-05

- Removed document structure section
- Add Azure guidance

12.7 2024-11

- GA Release

13 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

- Adrian Todorov
- Amber Reddy
- Andrei Burd
- Mark Lewis

- Prakash Manglanathan
- Shaun Coss